

comp10002

Foundations of Algorithms

Semester Two, 2025

Arrays and Algorithms

© The University of Melbourne, 2025
Lecture slides prepared by Alistair Moffat

Chapter 7

Chapter 12

Assertions

Efficiency

Binary search

Quicksort

Programs

Exercises

Chapter 7 (Part I)

Chapter 12 (Part I)

Assertions and correctness

Measuring efficiency

Binary search

Quicksort

Program examples

Exercises

Chapter 7

Chapter 12

Assertions

Efficiency

Binary search

Quicksort

Programs

Exercises

In an array, every element is of the same type, and the `i`'th value (counting from 0) can be accessed independently of other elements via `A[i]`. An array is an address mapping directly on to the computer's memory structure.

In K&R C, arrays must be sized at compilation time. In 1990 standard C, that restriction is lifted in functions – the array size may be an `int` function parameter.

There is **no** execution-time array bounds checking, and responsibility rests with the programmer.

Chapter 7

Chapter 12

Assertions

Efficiency

Binary search

Quicksort

Programs

Exercises

If used without a subscript, the array name itself is interpreted as being a **pointer constant**. When used in a function call (or anywhere else), the **address** of the array gets passed. The **size** of each element of the array must be declared, but not the **number** of elements.

In the function, the array argument is received as a **pointer variable**, and can be used to alter array elements. The notation `A[i]` is merely shorthand for `*(A+i)`.

Functions cannot return arrays, but (Chapter 10) are able to allocate new memory space and return a pointer to it.

Chapter 7

Chapter 12

Assertions

Efficiency

Binary search

Quicksort

Programs

Exercises

Chapter 7 – Program examples (Part I)

comp10002
Foundations of
Algorithms

lec05

Chapter 7

Chapter 12

Assertions

Efficiency

Binary search

Quicksort

Programs

Exercises

- ▶ `array1.c`
- ▶ `insertionsort.c`
- ▶ `twodarray.c`
- ▶ `matrixadd.c`
- ▶ `pointer4.c`

Exercise 1

Write a function that takes two arguments, an array `A` of type `int`, and an integer `n` indicating how many elements there are in `A`; and returns `IS_SORTED` if `A` is in (ascending) order, and `NOT_SORTED` if there are ordering violations.

Exercise 2

Ditto arguments, but returning the number of distinct values in the array `A`. You may not alter the array.

Exercise 3

Ditto arguments, but identifying the start point of the longest run of ascending values. You may not alter the array.

Key messages:

- ▶ Arrays are accessed by pointers; subscript indexing is just another form of pointer dereference
- ▶ Because of this relationship, there is no compile-time or run-time bounds checking
- ▶ Functions receive arrays as a pointer to an element of the array base type
- ▶ Two-dimensional (and higher) arrays need to be thought of as being arrays of arrays, including when being passed into functions.

Algorithms are at the heart of computing.

The most important attribute of any algorithm is correctness – it must solve the problem it claims to.

After that, we are interested in **resources** – the memory space required to execute it, and its execution time.

Both of these resource requirements are likely to grow as some function of n , where n is the “size” of the problem instance.

Given: an array A of some type, and an integer n indicating how many elements there are in A that may be inspected.

Question: does the element x (of the same type as the elements of A) appear in A ? If so, which location?

```
 $i \leftarrow 0$   
while  $i < n$  and  $A[i] \neq x$   
     $i \leftarrow i + 1$   
if  $i \geq n$   
    return not_found  
else  
    return  $i$ 
```

Can we demonstrate that this algorithm is correct?

Can we predict its behavior in terms of execution time?

Chapter 7

Chapter 12

Assertions

Efficiency

Binary search

Quicksort

Programs

Exercises

Assertions are argued statements about what must be true as a program executes.

With the use of logic, and precise rules associated with the semantics of executable statements, invariants can be used to provide formal proofs of program correctness.

Thinking in terms of invariants – especially loop invariants – will help you develop safe programs.

define $P \equiv (0 \leq i \leq n) \text{ and } (x \notin A[0 \dots i - 1])$

$i \leftarrow 0$

assert: P

while $i < n \text{ and } A[i] \neq x$

assert: $P \text{ and } (i < n \text{ and } A[i] \neq x)$

$i \leftarrow i + 1$

assert: P

assert: $P \text{ and } (i \geq n \text{ or } A[i] = x)$

[Chapter 7](#)[Chapter 12](#)[Assertions](#)[Efficiency](#)[Binary search](#)[Quicksort](#)[Programs](#)[Exercises](#)

reminder: $P \equiv (0 \leq i \leq n) \text{ and } (x \notin A[0 \dots i-1])$

assert: $P \text{ and } (i \geq n \text{ or } A[i] = x)$

if $i \geq n$

assert: $P \text{ and } (i \geq n \text{ or } A[i] = x) \text{ and } (i \geq n)$

$\implies x \notin A[0 \dots n-1]$

return *not_found*

else

assert: $P \text{ and } (i \geq n \text{ or } A[i] = x) \text{ and } (i < n)$

$\implies A[i] = x$

return i

Chapter 7

Chapter 12

Assertions

Efficiency

Binary search

Quicksort

Programs

Exercises

The C pseudo-function `assert` is defined in `assert.h` and can be inserted into your programs.

If the argument expression is violated, program execution will be halted and the invalid assertion identified.

You can do your `own` array bounds checking if you want to:

```
assert(0<=i && i<MAX);  
A[i] = A[i]+1;
```

Chapter 7

Chapter 12

Assertions

Efficiency

Binary search

Quicksort

Programs

Exercises

The number of loop iterations varies between zero and n .

In the **best case**, the item x is found in the first location.

But in the **worst case** (including all “*notfound*” outcomes), n loops iterations are required.

While it might be tempting to hope for the best, it is more professional (and much safer) to instead **plan for the worst**.

If the program requires $\approx c \cdot n$ computational steps on an input of size n , where c is a constant, it is **linear**.

If it takes 0.1 seconds when $n = 1,000,000$, it will take around 0.2 seconds if n is increased to 2,000,000.

Because computers have different speeds, it is the **rate of growth** that is important, not the exact running time on any input.

A different algorithm that requires $3n$ steps is still linear; as is a third method that requires $15n + 27\sqrt{n} - e \log n$ steps.

Chapter 7

Chapter 12

Assertions

Efficiency

Binary search

Quicksort

Programs

Exercises

To capture this idea, we define **sets** of functions that are all **asymptotically equivalent** in terms of their eventual long term growth rate:

$$f(n) \in O(g(n))$$

if and only if

$$\exists n_0, c > 0 : \forall n > n_0, f(n) \leq c \cdot g(n).$$

This is a complex definition!

[Chapter 7](#)[Chapter 12](#)[Assertions](#)[Efficiency](#)[Binary search](#)[Quicksort](#)[Programs](#)[Exercises](#)

In words (mathematical):

$f(n)$ is $O(g(n))$ when a positive threshold n_0 and a positive constant c can be identified such that, for every n larger than n_0 , $f(n)$ is bounded above by c times $g(n)$.

In words (practical):

$f(n)$ is $O(g(n))$ if, except for a constant factor, $g(n)$ grows at the same or a faster overall rate than $f(n)$ does, once n is large (enough).

Example 1: Consider $f_1(n) = 2n$.

Then $f_1(n) \in O(n)$.

And is also $O(2n - 1)$, $O(5n + \sqrt{n})$, $O(n \log n)$, and $O(n^2)$. And so on.

Example 2: Consider $f_2(n) = 15n + 27\sqrt{n} - e \log n$.

Then $f_2(n) \in O(n)$.

And is also $O(2n + \log n)$, $O(f_1(n))$, $O(n \log n)$, and $O(n^2)$. And so on.

[Chapter 7](#)[Chapter 12](#)[Assertions](#)[Efficiency](#)[Binary search](#)[Quicksort](#)[Programs](#)[Exercises](#)

Example 3: Take $f_3(n) = n(n+1)/2$.

Then $f_3(n) \in O(n^2)$.

And is also $O(5n^2 \log n + n^2)$ and $O(n^3)$.

Example 4: Take $f_4(n) = 100 + 0.01 \log n$.

Then $f_4(n) \in O(\log n)$, and $f_4(n) \notin O(1)$.

By definition, $O(g(n))$ is a **set** of functions, including all functions that have the same, or smaller, growth rate.

Given some function $f(n)$ to be categorized, it is usual to make use of the **simplest** such $g(n)$ function, by dropping constants and secondary terms; and also to choose $g(n)$ so that the **smallest** set $O(g(n))$ is generated.

So, while it is correct that $2n \log_2 n \in O(5n^2 - 3)$, it is usual to keep it simple, and say that $2n \log_2 n \in O(n \log n)$.

Why???

The “order” notation provides a tremendously useful abstraction that lets algorithms be compared.

If $f(n) \in O(g(n))$, and $g(n) \notin O(f(n))$, then an algorithm that requires $f(n)$ steps will, for large enough inputs, **always become faster** than an algorithm that takes $g(n)$ steps.

If array is *sorted*, can exit loop early:

```
 $i \leftarrow 0$   
while  $i < n$  and  $A[i] < x$   
     $i \leftarrow i + 1$   
if  $i \geq n$  or  $A[i] > x$   
    return not_found  
else  
    return  $i$ 
```

But worst-case execution time is still linear. And **average case** is only half that, so no better, **still** $O(n)$.

(Can you alter the previous predicate P and show that this method is also correct?)

[Chapter 7](#)[Chapter 12](#)[Assertions](#)[Efficiency](#)[Binary search](#)[Quicksort](#)[Programs](#)[Exercises](#)

If array is sorted, search by repeated range halving is **much** more efficient:

```
lo, hi  $\leftarrow 0, n$   
while lo < hi  
    m  $\leftarrow (lo + hi)/2$   
    if x < A[m]  
        hi  $\leftarrow m$   
    else if x > A[m]  
        lo  $\leftarrow m + 1$   
    else  
        return m  
return not_found
```

[Chapter 7](#)[Chapter 12](#)[Assertions](#)[Efficiency](#)[Binary search](#)[Quicksort](#)[Programs](#)[Exercises](#)

Correctness??

define $P \equiv (0 \leq lo \leq hi \leq n) \textbf{ and } (x \notin A[0 \dots lo - 1]) \textbf{ and } (x \notin A[hi \dots n - 1])$

Initialization:

$lo, hi \leftarrow 0, n$

assert: P

Loop:

while $lo < hi$

assert: $P \textbf{ and } lo < hi$

[Chapter 7](#)[Chapter 12](#)[Assertions](#)[Efficiency](#)[Binary search](#)[Quicksort](#)[Programs](#)[Exercises](#)

$$m \leftarrow (lo + hi)/2$$

assert: P **and** $lo \leq m < hi$

if $x < A[m]$

assert: P **and** $x < A[m \dots n - 1]$

$hi \leftarrow m$

assert: P

else if $x > A[m]$

assert: P **and** $x > A[0 \dots m]$

$lo \leftarrow m + 1$

assert: P

else

assert: P **and** $x \not< A[m]$ **and** $x \not> A[m]$
 $\implies x = A[m]$

return m

[Chapter 7](#)[Chapter 12](#)[Assertions](#)[Efficiency](#)[Binary search](#)[Quicksort](#)[Programs](#)[Exercises](#)

reminder: $P \equiv (0 \leq lo \leq hi \leq n) \text{ and } (x \notin A[0 \dots lo - 1]) \text{ and } (x \notin A[hi \dots n - 1])$

Upon loop exit:

assert: $P \text{ and } lo \geq hi$
 $\implies x \notin A[0 \dots n - 1]$

return *not_found*

It is also important to demonstrate that loops terminate. All the assertions in the world won't help if the loop doesn't actually exit to yield the required post-condition.

To show that a loop terminates:

1. Define an integer function $t()$ over suitable variables
2. Demonstrate that the loop invariant guarantees that $t() \leq t_0$ for some known value t_0
3. Demonstrate that every loop iteration reduces $t()$ by at least one
4. Demonstrate that the loop ends when $t() < 0$.

Chapter 7

Chapter 12

Assertions

Efficiency

Binary search

Quicksort

Programs

Exercises

In the case of binary search:

1. Define $t() = hi - lo$
2. Loop invariant means that $t_0 = n$
3. Two alternatives exist for loop iteration:
 - ▶ $hi \leftarrow m$, knowing $lo \leq m < hi$
 - ▶ $lo \leftarrow m + 1$, knowing $lo \leq m < hi$

Both alternatives reduce $t()$ by at least one

4. Loop only continues if $lo < hi$, that is, stops once $t() \leq 0$.

Don't underestimate the difficulty of getting even simple algorithms *exactly* right!

How long does binary search require?

Because of the halving process, the time taken is bounded by $T(n)$, where

$$T(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ 1 + T(\lfloor n/2 \rfloor) & \text{if } n > 1. \end{cases}$$

Recurrence relations often arise in the analysis of algorithms.

The exact solution of this one is $T(n) = 1 + \lfloor \log_2 n \rfloor \in O(\log n)$.

As a sanity check, suppose linear search takes (say) $2.5n$ fundamental (whatever that means) operations.

And that binary search, which is more complex, takes (say) $10 \log_2 n$ operations.

When is $2.5n < 10 \log_2 n$?

Only when $n < 16$.

The rapid superiority of binary search is unsurprising, since $\log n$ grows much more slowly than n .

Chapter 7

Chapter 12

Assertions

Efficiency

Binary search

Quicksort

Programs

Exercises

Binary search as a recursive C function

comp10002
Foundations of
Algorithms

lec05

Chapter 7

Chapter 12

Assertions

Efficiency

Binary search

Quicksort

Programs

Exercises

```
int
binary_search(data_t A[], int lo, int hi,
              data_t *key, int *locn) {
    int mid, outcome;
    /* if key is in A, it is between A[lo] and A[hi-1] */
    if (lo >= hi) {
        return BS_NOT_FOUND;
    }
    mid = (lo+hi)/2;
    if ((outcome = cmp(key, A+mid)) < 0) {
        return binary_search(A, lo, mid, key, locn);
    } else if (outcome > 0) {
        return binary_search(A, mid+1, hi, key, locn);
    } else {
        *locn = mid;
        return BS_FOUND;
    }
}
```


And in the calling program:

```
/* fill the array A[] with n items, in sorted order
   according to the comparison function cmp(), so that
   cmp(A+i-1, A+i) <= 0 for all values 0 < i < n */

key = /* some suitable value */

if (binary_search(A, 0, n, &key, &position) == BS_FOUND) {
    /* found, do something with A[position] */
} else {
    /* not found, do something else */
}
```

Chapter 7

Chapter 12

Assertions

Efficiency

Binary search

Quicksort

Programs

Exercises

The function `cmp` is a standard C paradigm.

It is usually passed in to a sorting or searching function as a **function argument** (Chapter 10).

It compares (via pointers to underlying objects) two elements, and returns:

- ▶ -ve, if first item should come prior to second
- ▶ 0, if two items can be considered to be equal
- ▶ +ve, if first item should come after second.

The string comparison functions `strcmp` and `strncmp` (Chapter 7, Part II) fit this template, as does the integer comparison function shown in `binarysearch.c`.

Chapter 7

Chapter 12

Assertions

Efficiency

Binary search

Quicksort

Programs

Exercises

How did the array get sorted in the first place?

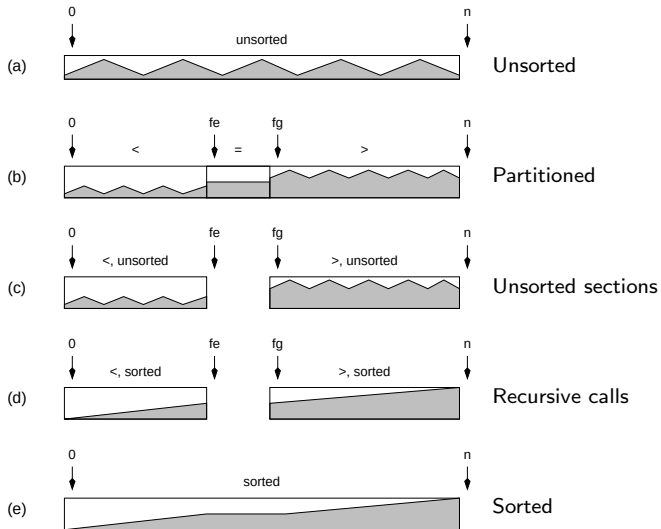
Chapter 7 describes [insertionsort](#).

In the worst case, insertionsort requires $n(n-1)/2$ comparisons between items. It is an $O(n^2)$ -time method. The time taken grows as a quadratic function of the size of the problem.

That is bad news, and insertionsort is not a good algorithm. Bubblesort, described in the yellow edition, has the same problem.

Does sorting *need* to take quadratic time?

Quicksort – The idea



Chapter 7

Chapter 12

Assertions

Efficiency

Binary search

Quicksort

Programs

Exercises

define $R \equiv 0 \leq fe < fg \leq n$ **and** $A[0 \dots fe - 1] < p$ **and**
 $A[fe \dots fg - 1] = p$ **and** $A[fg \dots n - 1] > p$

if $n \leq 1$

return

else

$p \leftarrow$ any element in $A[0 \dots n - 1]$

assert: $n > 1$ **and** $p \in A[0 \dots n - 1]$

$(fe, fg) \leftarrow \text{partition}(A, n, p)$

assert: R

$\text{quicksort}(A[0 \dots fe - 1])$

$\text{quicksort}(A[fg \dots n - 1])$

assert: $A[0 \dots fe - 1]$ is sorted **and** $A[fg \dots n - 1]$ is sorted
and $R \implies A[0 \dots n - 1]$ is sorted

[Chapter 7](#)[Chapter 12](#)[Assertions](#)[Efficiency](#)[Binary search](#)[Quicksort](#)[Programs](#)[Exercises](#)

Now have **specification** for *partition* – it permutes A in to three sections, $<$, $=$, and $>$ relative to pivot value p known to be initially present, and returns segment boundaries:

assert: $n > 1$ **and** $p \in A[0 \dots n - 1]$

$(fe, fg) \leftarrow \text{partition}(A, n, p)$

assert: R

PS. The assertions provided here don't require that the elements in A get permuted, so *partition* could in fact “cheat”, and assign complying values in to A .

define $P \equiv 0 \leq fe \leq next \leq fg \leq n$ **and** $A[0 \dots fe - 1] < p$ **and**
 $A[fe \dots next - 1] = p$ **and** $A[fg \dots n - 1] > p$ **and**
 $(p \in A[next \dots fg - 1] \text{ or } fe < next)$

Initialization:

$next, fe, fg \leftarrow 0, 0, n$

assert: P

Loop:

while $next < fg$

assert: P **and** $next < fg$

```
if  $A[next] < p$   
    swap  $A[fe]$  and  $A[next]$   
     $fe, next \leftarrow fe + 1, next + 1$   
    assert:  $P$   
else if  $A[next] > p$   
    swap  $A[next]$  and  $A[fg - 1]$   
     $fg \leftarrow fg - 1$   
    assert:  $P$   
else  
     $next \leftarrow next + 1$   
    assert:  $P$  and  $fe < next$ 
```

assert: P and $next \geq fg$ and $fe < next \implies R$

Chapter 7

Chapter 12

Assertions

Efficiency

Binary search

Quicksort

Programs

Exercises

To analyze execution time (and at the same time, to show termination), define $t() = fg - next$.

By the initial assignment, $t_0 = n$.

At every iteration, either $next$ goes up by one, or fg decreases by one. Therefore, $t()$ decreases by one at every iteration.

By the loop guard, the loop ends when $t() = 0$.

Hence, the number of iterations is never more than n , and so $partition \in O(n)$.

Hope for the best?

$$T(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ n + 2 \times T(\lfloor n/2 \rfloor) & \text{if } n > 1 \end{cases}$$

In this case, $T(n) \in O(n \log n)$. Wonderful...

(Actually, there is an even better situation, can you see it?)

Plan for the worst!

$$T(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ n + T(0) + T(n-1) & \text{if } n > 1 \end{cases}$$

Now $T(n) \in O(n^2)$.

Awful...

Is there something in between? Suppose the chosen pivot is equally likely to end up in any element in A . Then

$$T(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ n + \frac{2}{n} \sum_{i=0}^{n-1} T(i) & \text{if } n > 1 \end{cases}$$

Now what?

Turns out (after doing maths) that average case of Quicksort is also $O(n \log n)$, with the exact solution to the recurrence yielding

$$T(n) \approx 1.44n \log_2 n + O(n).$$

But, what does [average case](#) really mean?

Suppose $A[0 \dots n - 1]$ is a *random permutation* of final array. Then first item is also random. If so, can take:

$$p \leftarrow A[0]$$

This approach means that for the average case analysis to be valid, randomness is **required to be present in the input data**.

But if there is any chance at all of input being already sorted or nearly sorted or nearly reverse sorted, taking first item as pivot is a **high risk strategy**.

Better idea: Instead, take

$$p \leftarrow A[(n-1)/2]$$

Harder now to say what kind of sequence will force worst-case behavior. (But there are still such sequences.)

Or, could even be more defensive, and take

$$p \leftarrow \text{median}(A[0], A[(n-1)/2], A[n-1])$$

[Chapter 7](#)[Chapter 12](#)[Assertions](#)[Efficiency](#)[Binary search](#)[Quicksort](#)[Programs](#)[Exercises](#)

Brilliant idea: Instead, take

$$p \leftarrow A[\text{random}(0, n - 1)]$$

Now the randomness required for the analysis is **automatically supplied by the algorithm**.

The expected running time is $O(n \log n)$, *regardless* of the particular input sequence.

Types of analysis:

1. Best case: of academic interest only, only fools make use of it.
2. Average case:
 - 2a. Randomness required in input: high risk unless input can be guaranteed to be randomized.
 - 2b. Randomness enforced by algorithm, regardless of input: robust, but still not for true safety-critical applications.
3. Worst case: bet your (and others') life on it.

So, a question to ponder: can sorting be done in $O(n \log n)$ time in the **worst** case?

Chapter 7

Chapter 12

Assertions

Efficiency

Binary search

Quicksort

Programs

Exercises

Quicksort versus Quickpick?

	Quicksort	Quickpick \$20
Best case	$O(n)$	\$1,000,000
Average case	$O(n \log n)$	\$10
Worst case	$O(n^2)$	\$0

In Quicksort, the average is close to the **best**. Desirable outcome, and worth the risk.

In Quickpick, the average is close to the **worst**. Smarter to spend your money on something else.

Chapter 12 – Program examples

comp10002
Foundations of
Algorithms

lec05

Chapter 7

Chapter 12

Assertions

Efficiency

Binary search

Quicksort

Programs

Exercises

- ▶ `binarysearch.c`
- ▶ `quicksort.c`
- ▶ `sortscaffold.c`

Exercise 1

Suppose that the sorted array may contain duplicate items. Linear search will find the *first* match.

Modify the binary search algorithm so that it also identifies the first match if there are duplicates.

Modify the demonstration of correctness so that it matches your altered algorithm.

Exercise 2

A sorting algorithm is **stable** if the original array order is preserved among items with equal sort key.

2a. Show that Quicksort is not stable.

2b. How might it be modified so that it becomes stable?

Exercise 3

As shown, Quicksort requires two recursive calls.

But the final *tail recursion* can be replaced by inserting a while loop into the Quicksort function, leaving just one recursive call.

Implement this change, and test your implementation for correctness.

Exercise 4

A slightly different approach to *partition* is described by this more relaxed specification:

assert: $n > 1$ **and** $p \in A[0 \dots n - 1]$

$f \leftarrow \text{partition}(A, n, p)$

assert: $0 \leq f \leq n - 1$ **and** $A[0 \dots f - 1] \leq p$ **and**
 $A[f] = p$ **and** $A[f + 1 \dots n - 1] \geq p$

- 4a. Design a function that meets this specification.
- 4b. Does this approach have any disadvantages or advantages compared to the first one presented?

[Chapter 7](#)[Chapter 12](#)[Assertions](#)[Efficiency](#)[Binary search](#)[Quicksort](#)[Programs](#)[Exercises](#)

Exercise 5

Suppose that the k th smallest item is to be identified in an unsorted array $A[0 \dots n - 1]$.

- 5a. Design and analyze an algorithm for this problem that requires $O(nk)$ time in the *worst* case.
- 5b. Design and analyze an algorithm for this problem that requires $O(n)$ time in the *average* case.

You may not modify the array.

Exercise 6

Suppose that $A[0 \dots n - 1]$ is a sorted array, and that the position of x is to be found.

Suppose further that, although x might appear anywhere, it is often expected to be located relatively early in the array.

Design and analyze an algorithm for this problem that requires $O(\log d)$ time, where d is the index in A at which x (or the first element larger than it) appears, that is, $A[d - 1] < x \leq A[d]$.

Key messages:

- ▶ Algorithm correctness can be demonstrated via logic
- ▶ $O()$ notation allows asymptotic costs to be compared
- ▶ Best case analysis, and average case analysis where randomness is required in the input, are risky
- ▶ Worst case analysis, and average case analysis where randomness arises in the algorithm, are much better.
- ▶ Quicksort requires $O(n \log n)$ time, on average, but is $O(n^2)$ in the worst case.